nature methods

Perspective

https://doi.org/10.1038/s41592-024-02362-y

Guiding questions to avoid data leakage in biological machine learning applications

Received: 10 January 2024

Accepted: 26 June 2024

Published online: 9 August 2024

Check for updates

Judith Bernett[1,13], David B. Blumenthal[2,13] ✉, Dominik G. Grimm[3,4,5,13] ✉, Florian Haselbeck[3,4,6,13], Roman Joeres[7,8,9,10,13], Olga V. Kalinina[9,10,11,13] ✉ & Markus List[1,12,13] ✉

Machine learning methods for extracting patterns from high-dimensional data are very important in the biological sciences. However, in certain cases, real-world applications cannot confirm the reported prediction performance. One of the main reasons for this is data leakage, which can be seen as the illicit sharing of information between the training data and the test data, resulting in performance estimates that are far better than the performance observed in the intended application scenario. Data leakage can be difficult to detect in biological datasets due to their complex dependencies. With this in mind, we present seven questions that should be asked to prevent data leakage when constructing machine learning models in biological domains. We illustrate the usefulness of our questions by applying them to nontrivial examples. Our goal is to raise awareness of potential data leakage problems and to promote robust and reproducible machine learning-based research in biology.

Machine learning (ML) techniques have become indispensable for understanding complex biological systems. By applying these methods to biomolecular data, researchers identify patterns and relationships in, for example, DNA, RNA and protein sequences, enabling substantial advances in many areas of biological research[1–3]. However, numerous research articles indicate that the reported results of ML-based predictors are often overly optimistic and not reproducible on independent data[4–6]. This problem affects all ML-based research areas. However, it is particularly pressing in the life sciences, where the results of ML-based methods (or even the methods themselves) are ultimately intended to be translated into drugs, patient care, novel foods or chemical products. Failure to provide realistic estimates of true performance is, therefore, particularly critical and can even compromise patient safety or cause environmental and economic damage.

One of the main reasons for the discrepancy between reported and actual performance is data leakage—the illicit sharing of information between the data used to train and validate an ML model and the data used to test it, that is, to evaluate the model on unseen data[6–8]. When the information that is preserved between the training and the test data is not preserved along the transition from the training data to the data that is available once the ML model is deployed in its intended application scenario, the reported performance on the test data is usually not confirmed (Fig. 1). Instead, the model may use 'shortcuts' unavailable during its real-world application to achieve good performance on the

[1]TUM School of Life Sciences, Technical University of Munich, Freising, Germany. [2]Department Artificial Intelligence in Biomedical Engineering, Friedrich-Alexander-Universität Erlangen-Nürnberg, Erlangen, Germany. [3]TUM Campus Straubing for Biotechnology and Sustainability, Technical University of Munich, Straubing, Germany. [4]Bioinformatics, Weihenstephan-Triesdorf University of Applied Sciences, Straubing, Germany. [5]TUM School of Computation, Information and Technology, Technical University of Munich, Garching, Germany. [6]Smart Farming, Weihenstephan-Triesdorf University of Applied Sciences, Freising, Germany. [7]Department of Chemistry and Molecular Biology, University of Gothenburg, Gothenburg, Sweden. [8]Wallenberg Centre for Molecular and Translational Medicine, University of Gothenburg, Gothenburg, Sweden. [9]Helmholtz Institute for Pharmaceutical Research Saarland (HIPS), Helmholtz Centre for Infection Research (HZI), Saarbrücken, Germany. [10]Center for Bioinformatics, Saarland University, Saarbrücken, Germany. [11]Medical Faculty, Saarland University, Homburg, Germany. [12]Munich Data Science Institute (MDSI), Technical University of Munich, Garching, Germany. [13]These authors contributed equally: Judith Bernett, David B. Blumenthal, Dominik G. Grimm, Florian Haselbeck, Roman Joeres, Olga V. Kalinina, Markus List. ✉e-mail: david.b.blumenthal@fau.de; dominik.grimm@tum.de; olga.kalinina@helmholtz-hips.de; markus.list@tum.de

https://doi.org/10.1038/s41592-024-02362-y

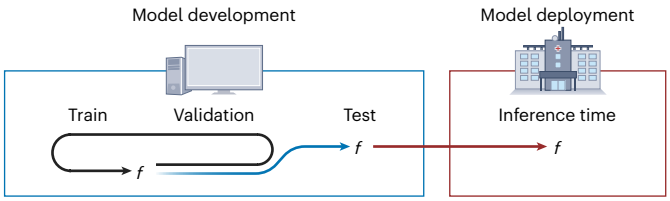


Fig. 1 | Visualization of the lifecycle of an ML model *f*. Data leakage occurs when information is preserved along the transition from training/validation to testing (blue arrow) but not from the development phase to the deployment scenario (red arrow).

test data. In biological domains, this problem is exacerbated because these shortcuts are often particularly subtle and difficult to identify.

This work presents seven questions designed to identify and avoid such pitfalls. We recommend that ML researchers working on biological applications consciously consider these questions when designing and evaluating their methods. For some questions, ML researchers without substantial domain knowledge in their application scenario may be unable to provide an informed answer. In such cases, we encourage them to engage in an interdisciplinary dialogue and consult domain experts. We demonstrate the utility of our question catalog through four supervised biological ML tasks: (1) predicting the pathogenicity of missense mutations, (2) predicting protein thermostability, (3) predicting protein–protein interactions and (4) predicting drug–target interactions. This article serves as a guide for biological researchers with a basic knowledge of ML and for ML researchers with a basic knowledge of biology. ML terms are explained in Table 1. By asking and reflecting on the questions posed in this Perspective, we hope that researchers will be able to identify data leakage problems that lead to overly optimistic performance estimates.

Defining 'data leakage'

Following the concepts of Kaufman et al.[7], we define data leakage as the relationship between (i) the reported performance scores of an ML-based predictor, (ii) the data used to train or validate the predictor (hereafter referred to as 'training data'), (iii) the data used to generate the reported performance scores of the ML-based predictor (hereafter referred to as 'test data' and (iv) the (possibly unseen) data on which the ML-based predictor should be run at inference time once it is deployed in its intended use case (hereafter referred to as 'inference-time data'):

Definition 1: Data leakage. Performance scores for an ML-based predictor *f* are inflated due to data leakage if *f* uses illegitimate information to achieve the reported scores, that is, information in the training data that generalizes to the test data but not to the inference-time data.

Definition 1 looks at data leakage from the broader perspective of the inference-time scenario. A taxonomy of data leakage that follows a similar approach has been proposed by Kapoor and Narayanan[6], who introduce three main errors that can lead to data leakage:

- "[L1] Lack of clean separation of train and test dataset"
- "[L2] Model uses features that are not legitimate"
- "[L3] Test set is not drawn from the distribution of scientific interest"

L1 is the narrower definition of data leakage. As the simplest special case of L1, Kapoor and Narayanan mention the lack of a hold-out test set for model evaluation on independent data (L1.1). Other examples include joint preprocessing (L1.2) and feature selection (L1.3) on training and test data (for example, joint normalization or clustering) or the existence of duplicates in the data, which are then used for both training and testing (L1.4).

Table 1 | Glossary of ML terms used within the scope of this paper

Term	Definition
Sample	A data point or entity (for example, a patient, measurement, protein) characterized by features.
Feature	Attribute of a data point (for example, age, sex, measurement unit, biochemical properties).
Objective	The task an ML model is optimized to predict. The objective function is either minimized or maximized during training.
Hold-out test set	A portion of the original dataset that is put aside during the training and validation process looked at only for the final model evaluation.
Inference	Using the model to make predictions for new data points in a realistic application scenario.
Preprocessing	The process of getting the initially obtained data into a shape and format in which it is ready for input into the model. This can include quality control, handling of missing values, normalization, encoding or embedding.
Normalization	Rescaling of the data, for example, to the range from 0 to 1, to account for differences between samples (for instance, the widely used TPM unit in RNA-sequencing quantification accounts for sequencing depth and gene length).
Feature selection	Selection of a subset of features used as input to the ML model. This process can be manual but should preferably be automized using performance on the validation dataset.
Encoding	Conversion of categorical or text input into numerical values (for example, one-hot encoding).
Embedding	Numerical representation of the data points. For example, a lower-dimensional representation could be obtained via dimensionality reduction or by pretraining a large language model.
Feature engineering	Extracting new, more effective features from the raw data points to feed into the ML model. Related to encoding and embedding.
Class imbalance	In a classification scenario, the target variable has multiple classes (categories of outcome like yes/no). When there are substantially overrepresented or underrepresented classes, we have a class imbalance.
Stratification	Stratification for imbalanced datasets ensures that the same proportion of each category occurs in the datasets, for example, for *k*-fold cross-validation.
Pretraining	The process of first training an ML model for a proxy task (for example, prediction of the next amino acid) and then using the output of this model (for example, as an embedding) as input to the actual model for the actual task (for example, PPI prediction).

L2 and L3 are part of our broader understanding of data leakage, that is, they influence the performance gap between test and inference time. While information from the test set is not explicitly used for training, it is implicitly responsible for the unrealistic performance measures. With respect to L2, a feature is illegitimate if it is only available during model training and evaluation but not at the time of inference. For example, using the prescription of an antihypertensive drug to predict hypertension[9] will result in near-perfect model performance during evaluation. However, the model will be useless because it can never be used to help diagnose new patients. Errors of the type L3 are further subdivided into dependencies between training and test data that do not exist between training and inference-time data (L3.2), temporal leakage in the evaluation of models for predicting future events (L3.1) and sampling bias, where the test data are sampled from a different distribution than the inference-time data (L3.3). Our broader

BOX 1

Relationship between data leakage and lack of reproducibility

According to Goodman et al.[76], three readings of the term 'reproducibility' must be distinguished:

- Method reproducibility: All resources (data, source code, detailed protocol of experimental setup, and so on) that are necessary for a study's re-implementation are publicly available.
- Result reproducibility: A very similar or identical experimental setup yields highly similar or identical results as reported in the original study.
- Inferential reproducibility: The qualitative conclusions drawn from a reanalysis or replication with new data are highly similar or identical to those drawn by the original study's authors.

Data leakage in the sense of definition 1 can lead to a lack of inferential reproducibility: When an ML-based predictor *f* uses information in the training data that generalizes to the test data but not to the inference-time data to achieve the performance scores reported in the original study, a reanalysis of data that more closely mirrors the envisaged inference-time use case may fail to support conclusions regarding *f*'s utility in that use case. The other two aspects of reproducibility are unrelated to data leakage.

understanding of data leakage is also closely related to the lack of reproducibility, another problem pervading biological ML research (Box 1).

There are at least two reasons why data leakage is particularly difficult to avoid in biological ML research. First, biological data typically exhibit complex dependency structures (for example, homology between nucleic or amino acid sequences or multiple genetic variants within the same gene[10]). These dependencies are often challenging for ML researchers to detect without substantial domain knowledge, potentially leading to L3.2-type leakage. Second, readily available biological data that can be used for training and testing are often biased toward well-studied molecules, for example, deleterious mutations in certain proteins[11] or overrepresentation of cancer-related proteins in protein–protein interaction networks[12,13]. In contrast, the intended inference-time use case of the trained ML models is often to predict properties of understudied molecules (a special case of L3.3).

Seven questions designed to reveal data leakage

We present seven questions designed to uncover illegitimate shortcuts and biases in the data that lead to inflated performance scores on the test set but prevent generalization at inference time. Figure 2 visually represents our seven questions. While these questions are not specific to biological applications of ML, they are especially difficult to detect in these settings.

Question 1 - Study bias: Are there overstudied or understudied elements in your dataset, and if so, how does this affect your model's performance at inference time?

Biomolecules and their modifications associated with well-studied diseases such as cancer, cardiovascular disease or Alzheimer's disease are overrepresented in many biological datasets. As a result, models may perform well on instances containing these overrepresented elements: If the test set contains mostly data for overrepresented biomolecules, the performance will appear impressive. If—at inference time—the model is applied to less-studied biomolecules, the reported test performances may be highly overoptimistic.

Question 2 - Label distribution: Are there peculiarities in the distributions of your training, validation and test labels that can lead to substantially better performance at test time than at inference time?

Biological data often show class imbalance. For example, there may be more healthy samples than disease samples, more non-binding pairs of molecules than binding ones, or more major than minor subvariants of a phenotype. ML requires balanced training datasets or weighted losses. Otherwise, the model may perform well by predicting only the majority class. Unbalanced training datasets may require undersampling the majority or oversampling the underrepresented class(es). An uninformed sampling strategy can create new shortcuts in predicting the label distributions, for example, if the sampling is done uniformly at random, but the major class(es) follow a different distribution. The performance may seem impressive when the model is evaluated on a test set containing the same biases. However, when a minority class sample is seen at inference time, the performance may be poor as the shortcut is not applicable.

Question 3 - Data shift: Is the dataset used for testing representative of the data to be used at inference time?

The key to recognizing data leakage is understanding the inference task, that is, what data the model should generalize to. This motivates partitioning the dataset into training, validation and test sets, where the latter should represent the (unknown) inference-time data. If the training and validation data contain shortcuts, that is, learning patterns that only apply to the used dataset, the learning problem appears easier than it is. Such shortcuts prevent the model from learning deeper features needed for generalization.

Question 4 - Inter-sample similarities: Are there (possibly hidden) similarities or dependencies between training and test samples that could lead to substantial differences in performance at test and inference time?

It is important to assess whether underlying similarities or dependencies exist between the training and test samples and if these or other similarities exist at inference time. Those similarities might be in sequence, structure, encoding/embedding space or other features leveraged by the model. Dependencies could be correlations between measurements of samples with respect to, for example, gene transcription, epigenetic modifications, or responses to unaccounted stimuli. An example is population structure, where models perform well within the same or similar populations but poorly when applied to a different population. These dependencies may be difficult to detect for humans, but machines can memorize the target variable for test samples from similar samples at training time. These similarities may not be present (or different) in the inference-time data. Therefore, the model will not perform well at inference time.

Question 5 - Unavailable features: Do the training and test data contain features not available at inference time?

It is important to avoid exploiting information that is unavailable for model development during inference, even if the information itself is valid. For biological data, avoiding this is often not trivial. An example is three-dimensional (3D) structures of proteins, which can be retrieved for wild types or well-studied mutants but may be inaccessible or expensive to obtain for artificially engineered or novel mutant sequences during inference. Information to build a prediction model must be carefully checked for its availability during inference.

Question 6 - Features as illegitimate surrogates: Does feature engineering affect performance differences between test and inference time, particularly if features serve as surrogates in train and test data but not at inference time?

Engineered features may describe general properties across multiple samples that are highly dataset specific and strongly correlate with the output label. In this case, there is a risk that these dependencies introduce information leakage and overly optimistic results. Even if the data splitting was performed correctly, the association of the engineered feature and outcome may no longer be present in new

https://doi.org/10.1038/s41592-024-02362-y

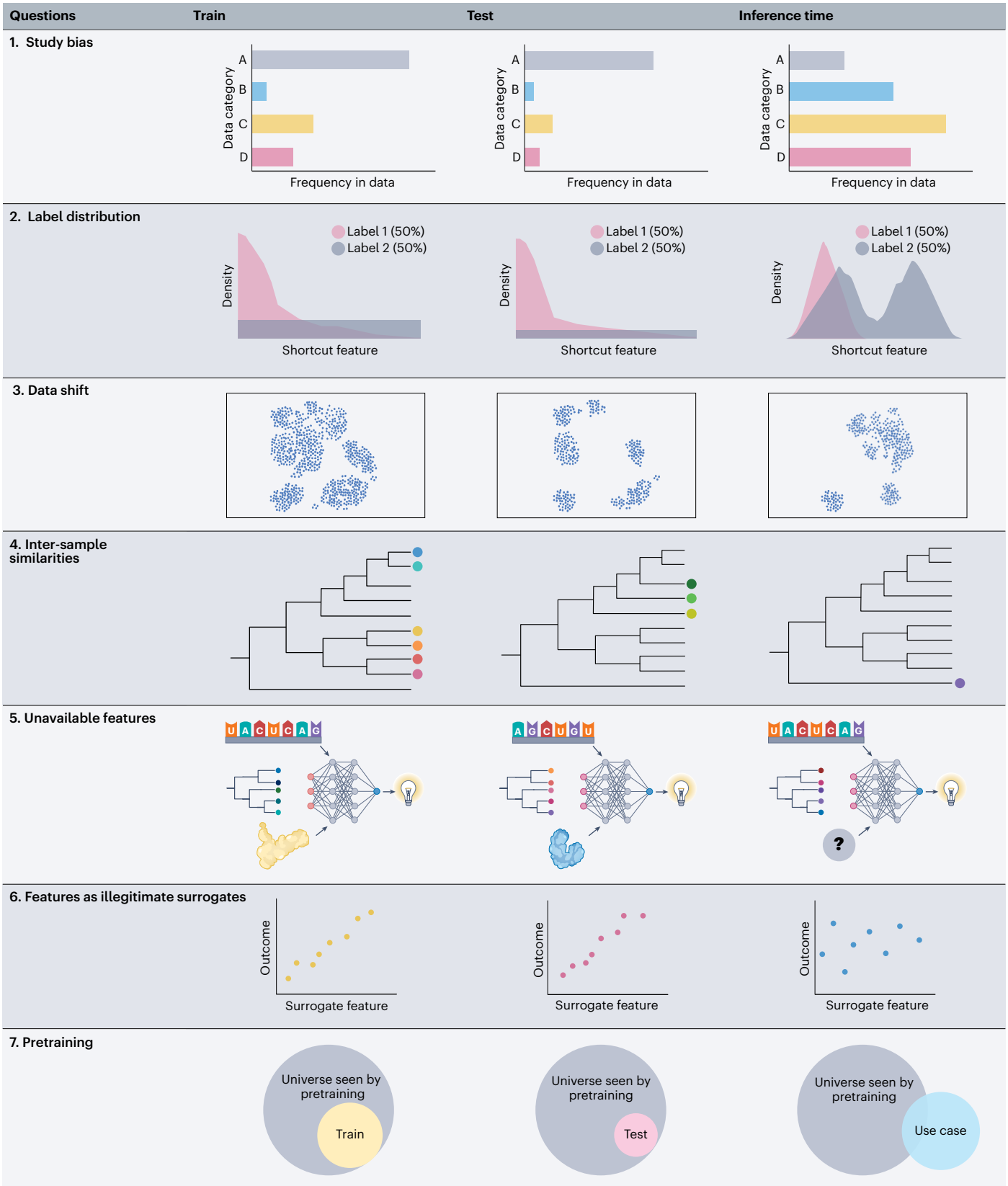


Fig. 2 | Schematic overview of the seven questions designed to reveal data leakage. The questions are intended to raise awareness for data leakage problems related to study bias, label distribution shifts, data shifts, inter-sample similarities, unavailability of features at inference time, usage of illegitimate surrogate features, and pretraining.

data samples. In addition, identifying this type of data leakage often requires domain-specific expert knowledge, for example, to exclude information about medications associated with a particular disease when trying to predict a patient's risk for that disease.

Question 7 - Pretraining: If one uses pretrained models, can information encoded in the pretrained embeddings lead to substantial differences in performance at test and inference time?

Embeddings obtained from large pretrained models, particularly large language models, gain popularity for biological ML tasks[14]. Since such embeddings can memorize information[15] (depending on the specific model architectures[16]), information encoded in the pretrained embeddings may lead to substantial differences in performance at test and inference time. Besides annotation leakage, similarities between the objectives used for pretraining and the downstream task contribute to this problem. Moreover, pretraining using a specific subset of instances might introduce data leakage through (dis-)similarities between the pretraining and test data, which do not extend to the inference-time data. To quantify such similarities, one can compare samples from the test set of the downstream prediction task to those used for pretraining. The impact on the performance of a predictor would then become apparent by pretraining an identical model, excluding samples that are similar to the samples of the downstream test set. However, this is computationally infeasible in most cases.

Detecting data leakage in four cases of biological ML tasks

We apply the presented question catalog to four biological ML tasks to illustrate the importance of domain knowledge and field-specific biases and pitfalls. Note that not all questions in our catalog are relevant in all application cases, as some challenges depend on the research question.

Task 1: Prediction of deleterious effects of missense mutations

Motivation and state of the art. Missense variants are single-nucleotide mutations that result in an amino acid change in encoded proteins, which can affect the solubility, stability and activity of proteins. Estimating the effect of missense variants is critical for many biotechnological and medical applications. Deleteriousness prediction of missense variants assesses whether a variant is likely to damage the encoded protein and is often used to study complex and Mendelian diseases[17]. While many prediction tools predict the deleterious effects of missense variants[18], the specific prediction target is diverse, for example, predicting sequence conservation, structural and functional effects, or the overall pathogenic potential of a variant[11]. Commonly used tools such as SIFT[19], PolyPhen2 (ref. 20), CADD[21] or AlphaMissense[10] assess deleteriousness either as a binary classification or by providing a probability score of the missense variant. However, the data used to train such predictors are highly affected by various types of hidden circularity, often leading to overly optimistic and spurious results[11].

Question 1 - Study bias: Certain genes and proteins within these datasets are overstudied due to their importance in specific diseases or biomolecular processes, resulting in a larger number of tagged mutations and the identification of substantially more deleterious mutations. Thus, existing variant databases are inherently biased as all variants of the same protein are often annotated as either neutral or deleterious. Classifiers that predict deleteriousness based on knowledge about specific variants in the same protein cannot predict novel deleterious variants within proteins with no previously annotated variants. Furthermore, they will also fail to discriminate between the two classes within the same protein[11].

Question 2 - Label distribution: Only a small fraction of proteins contain both deleterious and neutrally labeled variants and even more rarely in a balanced ratio. The data must be carefully stratified to avoid shifts in label distributions between training and inference. Thus, variants from the same protein should not be distributed across training, validation and testing[20]. Ideally, only proteins containing both types of variants are considered to avoid learning trivial majority vote-based predictors[11].

Question 3 - Data shift: Deleteriousness predictors must generalize to novel, unstudied proteins. However, if different variants of the same protein are distributed across training, validation and testing, general protein-specific information might be leaked between data splits. Similar to question 2, it is important to ensure that variants from the same protein are not distributed across different splits and that a protein stratification strategy is used[11,20].

Question 4 - Inter-sample similarities: Conservation of missense variants and evolutionary relatedness between proteins are highly correlated with the deleteriousness of certain variants. Thus, sequence conservation is often used to characterize common missense mutations in non-conserved positions as benign and rare mutations in conserved regions as deleterious, although this is not always true. Thus, the use of sequence homology and conservation scores as features may introduce additional biases when training such predictors and can lead to an increased number of false-positive predictions. The evolutionary conservation of variant effects also facilitates prediction when highly similar proteins are present in the training and test data. Special tools can be used to construct data splits that address this issue[22,23].

Question 5 - Unavailable features: Features derived from the 3D structure of proteins could be used to train deleteriousness prediction models. However, experimentally determined 3D structures may only be available for certain proteins and often only for the wild type or intensively studied mutations. Thus, the only option for obtaining such information during inference may be tools such as AlphaFold[1], RoseTTAFold[3] or Ember3D[24].

Question 6 - Features as illegitimate surrogates: The number of deleterious or neutral variants observed within a protein is a surrogate for identifying known disease-associated proteins (with many deleterious variants). Using such features will inflate performance compared to using them for proteins not yet associated with the disease at inference time, or for proteins with a mixture of deleterious and neutral variants[11].

Question 7 - Pretraining: Embeddings from pretrained protein language models[25–28] show promising results. However, sequences similar to the ones used for deleteriousness prediction have likely been seen during pretraining with unclear consequences for data leakage. While the pretraining task is usually different (for example, predicting masked amino acids) and does not use labels of the downstream application (for example, annotated variant effects), it has not been studied whether wild-type variants may be implicitly encoded in the embedding, which may simplify predicting the effect of point-wise mutations.

Task 2: Prediction of protein thermostability

Motivation and state of the art. The thermostability of a protein, that is, the resistance to changes in protein structure at elevated temperatures, is important in many areas of biotechnology, such as enzyme engineering. Accurately identifying the most promising variants among naturally occurring and engineered sequences is thus important. From an ML perspective, this can be seen as a regression or classification task, for example, predicting whether a protein will denature at certain temperatures or not, or whether a mutation will improve thermostability. Common approaches derive features such as physicochemical or structural properties from the amino acid sequences to apply classical ML-based prediction models, for example, support vector machines or ensemble learners[29–31]. Recently, methods using embeddings from protein language models have shown

superior performance for predicting protein thermophilicity (proteins in thermophilic species), a closely related prediction task[26,32,33]. However, several studies show that biased results are an issue in protein thermostability prediction due to various reasons[34–37].

Question 1 - Study bias: Researchers often test the effect of different amino acid substitutions on the same wild-type protein at the same or different positions. This can lead to datasets containing multiple mutants associated with the same wild type, but other less-studied wild types may be used to find new variants during inference. Thus, a bias toward mutants based on certain overrepresented wild-type proteins must be considered when developing a thermostability prediction model.

Question 2 - Label distribution: Publicly available protein thermostability datasets often contain more destabilizing and neutral than stabilizing variants[38]. This imbalance can be addressed with hypothetical reverse mutations, where the wild type is considered a stabilizing version of a destabilizing mutant, with the same label value as the forward mutation but the opposite sign. By utilizing this approach for data augmentation, it is possible to create a balanced dataset[39,40]. However, for prediction methods using the amino acid sequences, for example, via protein language models, this flipping can only be done once per wild type to prevent duplicates.

Question 3 - Data shift: Predicting protein thermostability or thermophilicity with mixed-species data could bias the performance estimate because some organisms live at higher temperatures and generally have more thermostable proteins. If the aim is predicting thermostability for the same species seen during training, it may be valid to neglect the species information. However, many real-world setups require predictions for unseen species or artificially engineered proteins, so proteins of the same species should not be split between training and test data.

Question 4 - Inter-sample similarities: Besides related species, the evolutionary relationship and sequence similarity of the proteins under study must be considered to avoid inflated performance measures, similar to the problem in task 1. Avoiding this issue in data splitting may require domain-specific knowledge and the use of bioinformatics tools to detect such similarities.

Question 5 - Unavailable features: The 3D structure of a protein is known to influence its thermostability. 3D information is often limited to wild type and well-studied mutant proteins (similar to task 1). Thus, often the only option to obtain structure information during inference is using tools such as AlphaFold[1], RoseTTAFold[3] or Ember3D[24]. However, these may be biased, as the predicted structures of well-studied proteins are likely of better quality.

Question 7 - **Pretraining**: Predictors using embeddings from protein language models have shown superior performance[26] but may introduce data leakage if pretraining was biased toward the test samples. Besides a completely different objective, the usual dataset sizes for thermostability prediction are tiny compared to those used for pretrained models, making it questionable that the pretraining was indeed biased toward the test samples. In a recent study, Haselbeck et al.[26] support this assumption by showing that a predictor based on a protein language model leads to similar performance drops on test data as other approaches that do not use pretrained models.

Task 3: Sequence-based protein–protein interaction prediction

Motivation and state of the art. Protein–protein interaction (PPI) networks are the basis for many biological downstream analyses, such as disease mechanism mining, signaling pathway analysis or protein function prediction[41,42]. Experimentally resolving every pairwise PPI is prohibitively expensive and laborious. For focused research questions, protein–protein docking software is available, but it requires structural information and is still too slow for high-throughput applications. For high-throughput problem settings, faster models predict PPIs from (the much more abundant) sequence data alone. Most methods consider PPI a binary prediction problem and the current state of the art uses protein language models to embed the protein sequences before processing them separately in multiple layers of a deep neural network. The deep representations are then combined for a final classifier. Several benchmarks highlight data leakage-related flaws in model training and evaluation of ML-based PPI predictors[43–46].

Question 1 - Study bias: Due to the bait–prey design of experimental methods to resolve PPIs (yeast two-hybrid, tandem affinity purification coupled with mass spectrometry), proteins that are often used as baits tend to emerge as hub nodes in PPI networks[13]. These might not be biologically plausible hubs (chaperones, proteins involved in transcription, translation, transport) but are subject to study bias where the focus is on disease-related proteins. Similarly, proteins not involved in common research questions become underrepresented in PPI networks. Hence, sequence-based PPI prediction methods are often trained and tested on interactions involving well-studied proteins. Because they just make up a small proportion of the proteome, performance measures might become inflated.

Question 2 - Label distribution: The points addressed in the previous question only concern positive examples (interacting proteins). For most ML settings, the model needs negative examples (non-interacting proteins). There are very few examples of curated negative interactions, for example, from the Negatome 2.0 database[47]. Thus, the number of curated examples is too low to allow for deep learning, and two alternative strategies are commonly used: (1) randomly pairing proteins for which no interaction is known and (2) randomly pairing proteins occurring in different subcellular locations for which no interaction is known. It has been shown that (2) renders the learning problem easier and leads to artificially inflated performance measures[48]. Furthermore, if the proteins in the negative examples do not have approximately the same node degree as in the positive ones, the model can use node degree as a shortcut to distinguish positive and negative examples, inflating prediction performance[43]. Consequently, when a candidate pair from the same subcellular compartment is presented at inference time, performance might drop sharply.

Question 3 - Data shift: Many methods randomly split the edges of their input network into training, validation and test sets. Hence, proteins from the test set were already seen during training, and information gathered about them can be used at test time. Randomly splitting up the edges is appropriate if we aim to predict missing links in the data, that is, completing the biological experiment that was not executed in an all-against-all fashion. However, if we seek to generalize to unseen or understudied proteins (the so-called dark proteome[49]), none of the proteins in the test set should be present in training. PPI prediction methods achieve excellent performance measures on random edge splits, whereas prediction performances become nearly random when leakage-aware edge split strategies are used[43].

Question 4 - Inter-sample similarities: For predicting interactions within the dark proteome (question 3), leaving the test proteins out of the training and validation datasets is insufficient to avoid leakage, as homologs tend to have similar interaction partners[50,51]. Additionally, sequence similarities between the proteins from the different sets should be minimized as the understudied proteins of the proteome often do not have any known homologs. Conversely, if we expect to learn patterns from homologs for, for example, cross-species prediction,

homologs of other species should be included in the test set. Importantly, the dataset split should account for node degrees to avoid methods that perform well on hub nodes only.

Question 7 - Pretraining: State-of-the-art deep learning models use residue-wise protein embeddings of protein language models as input for their first layer. These vary considerably in size and characteristics with implications for downstream prediction tasks. For instance, ESM-2 (ref. 52) was trained on over 60 million protein sequences from UniRef50. ProGen[53] was trained on over 280 million unique protein sequences, mainly obtained from UniParc. There are also smaller Word2Vec-based models relying on just a few thousand protein sequences[54,55]. PPI prediction based on simple models may not generalize to proteins unseen during training of the embedding even though it is likely that such a protein will be presented at inference time. For large pretrained models, it should be inspected whether the proteins seen during their training exclude a certain category of proteins. For instance, if the goal is to predict PPIs between artificial sequences, the embeddings of ESM-2—from which those sequences were explicitly excluded—might perform worse than the ones from ProGen, which was designed to predict artificial sequences as a downstream task.

Task 4: Prediction of drug–target interactions

Motivation and state of the art. Predicting drug–target interactions (also known as protein–ligand interactions) is a challenging problem in drug development. The objective is to qualitatively or quantitatively predict if or how strong an interaction between a protein and a small molecule (ligand, drug) is. The ML problem can be posed either as classification (where data points represent interactions or the absence thereof between a protein and a small molecule) or regression (where data points are continuous values, for example, IC_{50}, K_i or K_d). Datasets in this area typically represent interactions between two classes of chemically fundamentally different entities—proteins and small molecules—and can be regarded as two-dimensional matrices. Widely used datasets in this domain are Davis[56], KIBA[57], PDBBind[58] and BindingDB[59]. Concerns have been raised that data leakage issues limit the generalization capacity of existing ML models[60]. Several studies point to the discrepancy between the reported and actual performance of such models when applied to out-of-distribution data[22,61,62].

Question 1 - Study bias: Drug–target interaction datasets are generally biased toward well-studied proteins and drugs. Moreover, some datasets exclusively describe interactions for one family of proteins (for example, the KiBA[57] or Davis[56] datasets contain only kinases) and may not be useful for creating a model for other drug targets. As widely used datasets contain mostly small molecules, they may not be useful for models created for different types of biologically active molecules, such as natural products. Therefore, performance measures may be inflated by models that perform well on a few data points. For instance, it has recently been shown that topological shortcuts are present in the drug–target interaction data curated in BindingDB (proteins and small molecules with many data entries tend to be involved in positive interactions) and that state-of-the-art deep learning models leverage these shortcuts and, therefore, fail to generalize to unseen drugs or proteins[60].

Question 2 - Label distribution: For datasets such as KiBA[57] and Davis[56], regression labels are skewed toward weak and non-binding interactions because medicinal chemistry aims to identify selective binders. Conversely, PDBBind[58] and BindingDB[59] are not target specific, and their labels are skewed toward stronger interactions because interacting pairs are more likely reported. Moreover, protein targets often have only reported binders[60], making it very difficult for a model to predict non-binding molecules correctly (such examples exist in the G-protein-coupled-receptor-centered GLASS dataset[63]).

Question 3 - Data shift: A data shift between model development and inference time can be alleviated by sophisticated data splitting. If, at inference time, the model is expected to perform on proteins and/or drugs that are not part of the training dataset, performance measures will only be informative if the data are split accordingly into training, validation and test sets. Accepted splitting strategies in the field include splitting the sets of drugs or proteins and then assigning their interactions accordingly such that all interactions corresponding to one protein or drug are assigned to the same set (training, validation or test). Moreover, in so-called scaffold splits, interactions corresponding to small molecules with the same scaffold (and thus similar chemical properties) are assigned to the same set. Various methods for computing such biochemically informed splits exist (for example, DeepChem[64], TDCommons[65], GraphPart[23] or the LoHi splitter[66] for one-dimensional data splitting along the drug or target axis and DataSAIL[22] for simultaneous splitting along both axes).

Question 4 - Inter-sample similarities: Drug–target interaction datasets have high inter-sample similarities as most samples share the protein or the drug with other samples. Models may memorize a protein or drug from training as promiscuous, that is, likely to bind to any partner, which is not the desired outcome if specific and selective binders are sought. However, this may be a spurious effect caused by a high similarity of reported binding partners. To some extent, such effects can be accounted for by proper splitting, as described in question 3. Still, a certain prediction bias may be inevitable if the corresponding training data are unavailable (for example, measurements have been made only for a set of highly similar compounds). In this case, data augmentation strategies can be considered.

Question 5 - Unavailable features: Models for predicting drug–target interactions may rely on either the sequence of the drug target or its 3D structure. In the latter case, as in several other tasks above, special care should be taken whether experimentally resolved or predicted structures are used. More specifically, for the prediction of drug–target interactions, a very valuable piece of information is an experimentally resolved target structure with any ligand bound. The specific arrangement of amino acid side chains in the binding pocket and a possible overall change of protein conformation upon binding can provide additional information for some targets, which is unavailable for others.

Question 7 - Pretraining: In drug–target interaction prediction, recent ML methods rely on pretrained embeddings from both protein language models (for example, ESM-2 (ref. 52), Ankh[67] or ProtTrans[25]) and chemical foundation models such as ChemBERTa[68]. It is likely that datasets used for training and testing the ML-based drug–target interaction predictors contain proteins and chemicals seen during pretraining. This may lead to data leakage, especially via the embeddings of chemicals: In the downstream application at inference time (for example, drug discovery), the ML models are often used to predict binding affinities for novel classes of compounds that are not included in databases such as PubChem[69] and, therefore, were not used for pretraining of the chemical foundation models.

Conclusions

In biological research, the use of artificial intelligence has created countless possibilities and opportunities. However, using ML-based approaches has several pitfalls that can quickly lead to data leakage and, thus, to reporting overly optimistic results. Avoiding such pitfalls requires expertise in both ML and the biological application domain. Nowadays, ensuring a valid ML workflow has become more feasible thanks to popular frameworks such as PyTorch[70], TensorFlow[71] and scikit-learn[72]. In practice, their ease of use quickly increases the risk of scientifically incorrect application and spurious results. Conversely, the complexity of biological data can lead to data leakage if overseen

by data scientists who do not have sufficient qualifications in the respective application domain. For these reasons, we highly encourage interdisciplinary collaboration between experts in both fields. With the seven questions above, we aim to make researchers aware of the complex latent interdependencies and possibilities of data leakage in biological applications and to provide a guideline for potential issues to consider in interdisciplinary collaborations.

Glossary of ML terms

The glossary in Table 1 explains ML terms used in this paper. For a deeper understanding of ML and deep learning, we refer to Hastie et al.[73] and Goodfellow et al.[74]. Ching et al.[75] provide an excellent overview of ML trends in biology and medicine.

Data availability

No data were generated or analyzed for this work, and no source code was developed.

References

1. Jumper, J. et al. Highly accurate protein structure prediction with alphafold. *Nature* **596**, 583–589 (2021).
2. Avsec, Ž. et al. Effective gene expression prediction from sequence by integrating long-range interactions. *Nat. Methods* **18**, 1196–1203 (2021).
3. Baek, M. et al. Accurate prediction of protein structures and interactions using a three-track neural network. *Science* **373**, 871–876 (2021).
4. Gundersen, O. E. & Kjensmo, S. State of the art: reproducibility in artificial intelligence. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 32, No. 1 https://ojs.aaai.org/index.php/AAAI/article/view/11503 (2018).
5. Matschinske, J. et al. The AIMe registry for artificial intelligence in biomedical research. *Nat. Methods* **18**, 1128–1131 (2021).
6. Kapoor, S. & Narayanan, A. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns* **4**, 100804 (2023).
This article presents a taxonomy of common pitfalls that introduce data leakage and lead to overoptimistic results in many scientific fields. The authors also suggest model info sheets to identify and prevent those pitfalls and, ultimately, counteract the reproducibility crisis.
7. Kaufman, S., Rosset, S., Perlich, C. & Stitelman, O. Leakage in data mining: formulation, detection, and avoidance. *ACM Trans. Knowl. Discov. Data* **6**, 1–21 (2012).
This article provides a formal definition of data leakage and suggests ways to detect and avoid it.
8. Whalen, S., Schreiber, J., Noble, W. S. & Pollard, K. S. Navigating the pitfalls of applying machine learning in genomics. *Nat. Rev. Genet.* **23**, 169–181 (2022).
9. Chiavegatto Filho, A., Batista, A. F. D. M. & Dos Santos, H. G. Data leakage in health outcomes prediction with machine learning. Comment on 'prediction of incident hypertension within the next year: prospective study using statewide electronic health records and machine learning'. *J. Med. Internet Res.* **23**, e10969 (2021).
10. Cheng, J. et al. Accurate proteome-wide missense variant effect prediction with alphamissense. *Science* **381**, eadg7492 (2023).
11. Grimm, D. G. et al. The evaluation of tools used to predict the impact of missense variants is hindered by two types of circularity. *Hum. Mutat.* **36**, 513–523 (2015).
This article demonstrates two types of circularity that lead to overly optimistic results for deleteriousness prediction tools.
12. Schaefer, M. H., Serrano, L. & Andrade-Navarro, M. A. Correcting for the study bias associated with protein–protein interaction measurements reveals differences between protein degree distributions from different cancer types. *Front. Genet.* **6**, 137790 (2015).
13. Lucchetta, M., List, M., Blumenthal, D. B. & Schaefer, M. H. Emergence of power-law distributions in protein–protein interaction networks through study bias. Preprint at *bioRxiv* https://doi.org/10.1101/2023.03.17.533165 (2023).
14. Ofer, D., Brandes, N. & Linial, M. The language of proteins: Nlp, machine learning & protein sequences. *Comput. Struct. Biotechnol. J.* **19**, 1750–1758 (2021).
15. Song, C. & Raghunathan, A. Information leakage in embedding models. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, 377–390 (2020).
16. Zhang, G. et al. How does a deep learning model architecture impact its privacy? a comprehensive study of privacy attacks on CNNs and transformers. Preprint at https://arxiv.org/abs/2210.11049 (2022).
17. Rentzsch, P., Witten, D., Cooper, G. M., Shendure, J. & Kircher, M. CADD: predicting the deleteriousness of variants throughout the human genome. *Nucleic Acids Res.* **47**, D886–D894 (2019).
18. Notin, P. et al. ProteinGym: large-scale benchmarks for protein design and fitness prediction. In *Advances in Neural Information Processing Systems* **36** (NeurIPS, 2023).
19. Ng, P. C. & Henikoff, S. SIFT: predicting amino acid changes that affect protein function. *Nucleic Acids Res.* **31**, 3812–3814 (2003).
20. Adzhubei, I. A. et al. A method and server for predicting damaging missense mutations. *Nat. Methods* **7**, 248–249 (2010).
21. Kircher, M. et al. A general framework for estimating the relative pathogenicity of human genetic variants. *Nat. Genet.* **46**, 310–315 (2014).
22. Joeres, R., Blumenthal, D. B. & Kalinina, O. V. Datasail: data splitting against information leakage. Preprint at *bioRxiv* https://doi.org/10.1101/2023.11.15.566305 (2023).
23. Teufel, F. et al. GraphPart: homology partitioning for biological sequence analysis. *NAR Genom. Bioinform.* **5**, lqad088 (2023).
24. Weissenow, K., Heinzinger, M., Steinegger, M. & Rost, B. Ultra-fast protein structure prediction to capture effects of sequence variation in mutation movies. Preprint at *bioRxiv* https://doi.org/10.1101/2022.11.14.516473 (2022).
25. Elnaggar, A. et al. ProtTrans: toward understanding the language of life through self-supervised learning. *IEEE Trans. Pattern Anal. Mach. Intell.* **44**, 7112–7127 (2021).
26. Haselbeck, F. et al. Superior protein thermophilicity prediction with protein language model embeddings. *NAR Genom. Bioinform.* **5**, lqad087 (2023).
27. Teufel, F. et al. Signalp 6.0 predicts all five types of signal peptides using protein language models. *Nat. Biotechnol.* **40**, 1023–1025 (2022).
28. Wu, R. et al. High-resolution de novo structure prediction from primary sequence. Preprint at *bioRxiv* https://doi.org/10.1101/2022.07.21.500999 (2022).
29. Charoenkwan, P. et al. SAPPHIRE: a stacking-based ensemble learning framework for accurate prediction of thermophilic proteins. *Comput. Biol. Med.* **146**, 105704 (2022).
30. Lin, H. & Chen, W. Prediction of thermophilic proteins using feature selection technique. *J. Microbiol. Methods* **84**, 67–70 (2011).
31. Ahmed, Z. et al. iThermo: a sequence-based model for identifying thermophilic proteins using a multi-feature fusion strategy. *Front. Microbiol.* **13**, 790063 (2022).
32. Pei, H. et al. Identification of thermophilic proteins based on sequence-based bidirectional representations from transformer-embedding features. *Appl. Sci.* **13**, 2858 (2023).
33. Pudžiuvelytė, I. et al. TemStaPro: protein thermostability prediction using sequence representations from protein language models. *Bioinformatics* **40**, btae157 (2024).

34. Pucci, F., Bernaerts, K. V., Kwasigroch, J. M. & Rooman, M. Quantification of biases in predictions of protein stability changes upon mutations. *Bioinformatics* **34**, 3659–3665 (2018).
This article analyzes biases in protein stability prediction tools and shows that most predictors favor destabilizing mutations. The authors also propose a new method addressing this issue by imposing physical symmetries under inverse mutations.
35. Usmanova, D. R. et al. Self-consistency test reveals systematic bias in programs for prediction change of stability upon mutation. *Bioinformatics* **34**, 3653–3658 (2018).
36. Fang, J. The role of data imbalance bias in the prediction of protein stability change upon mutation. *PLoS ONE* **18**, e0283727 (2023).
37. Sanavia, T. et al. Limitations and challenges in protein stability prediction upon genome variations: towards future applications in precision medicine. *Comput. Struct. Biotechnol. J.* **18**, 1968–1979 (2020).
38. Stourac, J. et al. Fireprotdb: database of manually curated protein stability data. *Nucleic Acids Res.* **49**, D319–D324 (2021).
39. Rodrigues, C. H., Pires, D. E. & Ascher, D. B. Dynamut2: Assessing changes in stability and flexibility upon single and multiple point missense mutations. *Protein Sci.* **30**, 60–69 (2021).
40. Fang, J. A critical review of five machine learning-based algorithms for predicting protein stability changes upon mutation. *Brief. Bioinform.* **21**, 1285–1292 (2019).
41. Menche, J. et al. Uncovering disease-disease relationships through the incomplete interactome. *Science* **347**, 1257601 (2015).
42. Batra, R. et al. On the performance of de novo pathway enrichment. *NPJ Syst. Biol. Appl.* **3**, 6 (2017).
43. Bernett, J., Blumenthal, D. B. & List, M. Cracking the black box of deep sequence-based protein–protein interaction prediction. *Brief. Bioinform.* **25**, bbae076 (2024).
This article shows that reported performances of numerous deep learning-based protein–protein interaction prediction models are massively inflated due to data leakage. The authors also provide a leakage-free gold-standard dataset to foster the development of better protein–protein interaction predictors in the future.
44. Park, Y. & Marcotte, E. M. Flaws in evaluation schemes for pair-input computational predictions. *Nat. Methods* **9**, 1134–1136 (2012).
45. Dunham, B. & Ganapathiraju, M. K. Benchmark evaluation of protein–protein interaction prediction algorithms. *Molecules* **27**, 41 (2021).
46. Hamp, T. & Rost, B. Evolutionary profiles improve protein–protein interaction prediction from sequence. *Bioinformatics* **31**, 1945–1950 (2015).
47. Blohm, P. et al. Negatome 2.0: a database of non-interacting proteins derived by literature mining, manual annotation and protein structure analysis. *Nucleic Acids Res.* **42**, D396–D400 (2014).
48. Ben-Hur, A. & Noble, W. S. Choosing negative examples for the prediction of protein–protein interactions. *BMC Bioinformatics* **7**, S2 (2006).
49. Tabar, M. S. et al. Illuminating the dark protein–protein interactome. *Cell Rep. Methods* **2**, 100275 (2022).
50. Aloy, P., Ceulemans, H., Stark, A. & Russell, R. B. The relationship between sequence and interaction divergence in proteins. *J. Mol. Biol.* **332**, 989–998 (2003).
51. Marsh, J. A. & Teichmann, S. A. Structure, dynamics, assembly, and evolution of protein complexes. *Annu. Rev. Biochem.* **84**, 551–575 (2015).
52. Lin, Z. et al. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science* **379**, 1123–1130 (2023).
53. Madani, A. et al. Large language models generate functional protein sequences across diverse families. *Nat. Biotechnol.* **41**, 1099–1106 (2023).
54. Yao, Y., Du, X., Diao, Y. & Zhu, H. An integration of deep learning with feature embedding for protein–protein interaction prediction. *PeerJ* **7**, e7126 (2019).
55. Chen, M. et al. Multifaceted protein–protein interaction prediction based on Siamese residual RCNN. *Bioinformatics* **35**, i305–i314 (2019).
56. Davis, M. I. et al. Comprehensive analysis of kinase inhibitor selectivity. *Nat. Biotechnol.* **29**, 1046–1051 (2011).
57. Tang, J. et al. Making sense of large-scale kinase inhibitor bioactivity data sets: a comparative and integrative analysis. *J. Chem. Inf. Model.* **54**, 735–743 (2014).
58. Liu, Z. et al. PDB-wide collection of binding data: current status of the PDBbind database. *Bioinformatics* **31**, 405–412 (2015).
59. Liu, T., Lin, Y., Wen, X., Jorissen, R. N. & Gilson, M. K. BindingDB: a web-accessible database of experimentally determined protein–ligand binding affinities. *Nucleic Acids Res.* **35**, D198–D201 (2007).
60. Chatterjee, A. et al. Improving the generalizability of protein–ligand binding predictions with AI-Bind. *Nat. Commun.* **14**, 1989 (2023).
This article shows how deep learning models for drug–target interaction prediction learn shortcuts from the topology of the training network instead of hidden mechanisms and, hence, generalize poorly. The authors further propose a new method designed to overcome these shortcomings.
61. Bai, P. et al. Hierarchical clustering split for low-bias evaluation of drug–target interaction prediction. In *2021 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)*, 641–644 (IEEE, 2021).
62. Torrisi, M., de la Vega de León, A., Climent, G., Loos, R. & Panjkovich, A. Improving the assessment of deep learning models in the context of drug–target interaction prediction. Preprint at *bioRxiv* https://doi.org/10.1101/2022.04.20.488898 (2022).
63. Chan, W. K. et al. GLASS: a comprehensive database for experimentally validated GPCR–ligand associations. *Bioinformatics* **31**, 3035–3042 (2015).
64. Ramsundar, B. *Molecular machine learning with DeepChem*. Ph.D. thesis, Stanford University (2018).
65. Huang, K. et al. Artificial intelligence foundation for therapeutic science. *Nat. Chem. Biol.* **18**, 1033–1036 (2022).
66. Steshin, S. Lo-Hi: practical Ml drug discovery benchmark. In *Advances in Neural Information Processing Systems* **36** (NeurIPS, 2023).
67. Elnaggar, A. et al. Ankh: optimized protein language model unlocks general-purpose modelling. Preprint at https://arxiv.org/abs/2301.06568 (2023).
68. Chithrananda, S., Grand, G. & Ramsundar, B. ChemBERTa: large-scale self-supervised pretraining for molecular property prediction. Preprint at https://arxiv.org/abs/2010.09885 (2020).
69. Kim, S. et al. Pubchem 2019 update: improved access to chemical data. *Nucleic Acids Res.* **47**, D1102–D1109 (2019).
70. Paszke, A. et al. PyTorch: an imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems* **32** (NeurIPS, 2019).
71. Abadi, M. et al. TensorFlow: large-scale machine learning on heterogeneous systems. In *12th USENIX Symposium on Operating Systems Design and Implementation* (USENIX, 2016).
72. Pedregosa, F. et al. Scikit-learn: machine learning in Python. *J. Mach. Learn. Res.* **12**, 2825–2830 (2011).
73. Hastie, T., Tibshirani, R. & Friedman, J. H. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, vol. 2 (Springer, 2009).

74. Goodfellow, I., Bengio, Y. & Courville, A. *Deep Learning* (MIT Press, 2016); http://www.deeplearningbook.org/
75. Ching, T. et al. Opportunities and obstacles for deep learning in biology and medicine. *J. R. Soc. Interface* **15**, 20170387 (2018).
76. Goodman, S. N., Fanelli, D. & Ioannidis, J. P. A. What does research reproducibility mean? *Sci. Transl. Med.* **8**, 341ps12 (2016).
This article provides a subdivision of the term 'reproducibility' into 'methods reproducibility', 'results reproducibility' and 'inferential reproducibility'. Data leakage is one important source of lack of inferential reproducibility.

Acknowledgements

J.B. and M.L. were supported by the German Federal Ministry of Education and Research (BMBF) within the framework of the CompLS funding concept (031L0305A, DROP2AI). M.L. was additionally funded by the Deutsche Forschungsgemeinschaft (German Research Foundation; 422216132). D.B.B. was supported by the BMBF within the framework of the CompLS funding concept (031L0309A, NetMap). M.L. and D.B.B. were funded by the Deutsche Forschungsgemeinschaft (German Research Foundation; 516188180). R.J. was supported by the HelmholtzAI grant XAI-Graph, the Knut and Alice Wallenberg Foundation and the University of Gothenburg. O.V.K. acknowledges financial support from the Klaus Faber Foundation. R.J. and O.V.K. thank A. Gress and I. Senatorov for fruitful discussions.

Author contributions

J.B., D.B.B., D.G.G., F.H., R.J., O.V.K. and M.L. jointly initiated this work, conceptualized the used understanding of data leakage, phrased the seven guiding questions, and wrote and revised the manuscript. D.G.G. and F.H. provided the details for the deleteriousness prediction and thermostability prediction use cases. D.B.B., J.B. and M.L. provided the details for the protein–protein interaction prediction use case. R.J. and O.V.K. provided the details for the drug–target interaction prediction use case. R.J. designed Fig. 1. J.B. designed Fig. 2. D.B.B. provided the details for the link between data leakage and lack of reproducibility.

Competing interests

M.L. consults for mbiomics. D.B.B. consults for BioVariance. All other authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to David B. Blumenthal, Dominik G. Grimm, Olga V. Kalinina or Markus List.

Peer review information *Nature Methods* thanks Mark Craven, Arunima Singh and the other, anonymous, reviewer(s) for their contribution to the peer review of this work.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

© Springer Nature America, Inc. 2024